PRD: Master Data Hub Platform (Semarchy xDM-class)

Version: 2.0 | Date: 2026-02-17 | Status: Comprehensive draft for build kickoff

1. Executive Summary

We will build a model-driven Master Data Management platform that creates governed, certified, and AI-ready master data for enterprise domains (customer, product, supplier, location, reference). The platform must support the full certification lifecycle: ingest -> validate -> standardize/enrich -> match/merge -> stewardship workflow -> consolidate -> publish -> notify -> monitor. The product is designed to be Semarchy xDM-class in capabilities and usability, with an API-first and extensible architecture.

1.1 Objectives

- Single source of truth via golden records with full lineage, audit, and explainability.
- Business-led stewardship through low-code UI, workflow tasks, and guided duplicate management.
- Multi-domain modeling with reference data, hierarchies, and relationship graphs.
- API-first publishing (REST + SQL) plus custom endpoints (named queries).
- Extensibility through plugins (validators/enrichers) including AI enrichers/classifiers.
- Deployment flexibility: self-hosted/containerized, cloud SaaS, and future warehouse-native execution.

1.2 Target Users and Buying Centers

- Chief Data Officer / Data Governance leads (trust, compliance, governance).
- Domain data owners (customer/product/supplier) (policy and stewardship).
- Data engineering / integration teams (publishing and operationalization).
- Security and compliance teams (audit, privacy, access control).

1.3 MVP Definition

MVP is a complete end-to-end demo and production pilot for one domain (e.g., Customer) with: model designer, schema generation, batch loads, validations, basic enrichment, browse/search UI, audit/history, and API consumption. Matching/merge, named queries, workflows, and notifications are added by v1.0.

1.4 Out of Scope for MVP

- Full ETL/ELT replacement; we integrate with external pipelines.

- Streaming ingestion for every protocol; prioritize batch/REST/SQL for MVP.
- Warehouse-native execution; treat as v1.2+ roadmap.

2. Problem Statement and Value Proposition

2.1 Problem

Enterprises suffer from duplicate, inconsistent, and stale master data across systems. Downstream analytics, operations, and AI initiatives fail because master data is not certified, and because ownership, stewardship, and auditability are unclear.

2.2 Value

- Reduce duplicates and incorrect joins across systems, improving operational accuracy.
- Increase speed of decision-making by providing a trusted golden record with provenance.
- Enable compliance through audit trails, record-level permissions, and controlled publishing.
- Decrease integration complexity with standard APIs and repeatable certification pipelines.

3. Personas and Stakeholders

Persona	Primary Goals	Key Actions	Success Signals
Business User	Find trusted data and context	Search/browse, follow hierarchies/relations hips	Fewer disputes; fewer manual reconciliations
Data Steward	Resolve duplicates and exceptions	Manage duplicates, fix errors, approve workflows	Lower duplicate rate; SLA compliance
Data Owner	Define standards and policies	Approve rules, reference lists, access policies	Audit pass; policy adherence
Integration Developer	Publish/consume data reliably	Build pipelines, create named queries, manage loads	Stable APIs; predictable loads
Platform Admin	Operate and secure platform	Tenants, upgrades, monitoring, users/roles	Low downtime; controlled changes
Data Engineer /	Design models and	Model entities; tune	High match precision;

Architect	pipelines	match/survivorship; performance	scalable jobs

4. Key User Journeys

4.1 Journey A - Integrate and certify a new data source

1. Integration dev maps source fields to entity attributes and chooses load strategy (batch, incremental, or API-driven).
2. Source records are loaded with a Load ID; pre-validation runs and errors are isolated for correction.
3. Enrichers standardize formats (names, addresses, phone, email) and compute derived attributes.
4. Matching runs to link source records to existing masters; new masters are created as needed.
5. Golden record is produced; changes are visible in UI and via APIs; downstream is notified (v1.0+).

4.2 Journey B - Resolve duplicates and approve a change

6. Steward opens a duplicate queue; reviews grouped candidates with match explanations and key differences.
7. Steward merges records (or splits) and optionally overrides specific attributes with justification.
8. A workflow task is created for approval if policy requires (e.g., high-impact changes).
9. Once approved, consolidation runs and golden record updates; full audit trail is preserved.

4.3 Journey C - Publish certified data to downstream systems

10. App/integration consumes GD (golden) data via REST/SQL for standard use cases.
11. Named queries provide domain-specific endpoints (e.g., customer + addresses + contacts) without custom coding.
12. Exports (CSV) and notification triggers keep downstream caches synced.

5. End-to-End Product Flow (Comprehensive)

5.1 Design-time: Build the Data Hub

13. Model Designer: define domains, entities, attributes (simple/complex types), relationships, and naming rules.
14. Entity Strategy: choose basic vs ID-matched vs fuzzy-matched per entity; configure keys and external identifiers.
15. Rule Builder: define validators, enrichers, match rules, survivorship strategies, and override policies.

16. App Builder: generate and customize business views, forms, cards, collections; configure navigation across relationships; create dashboards.
17. Workflow Builder: define stewardship workflows with conditional routing; SLAs; escalation; notifications.
18. API & Access: expose entities and named queries; define scopes and permissions (role + record + attribute level).
19. Deployment/Promotion: publish model version to environment; generate migrations; run smoke checks.

5.2 Run-time: Certification Lifecycle

20. Ingest: publishers load records via SQL API / REST / UI import; each batch tracked with Load ID and timestamps.
21. Pre-validation: validate records; failed records go to error stores for recycling; create stewardship tasks if required.
22. Enrichment: standardize and augment records (including AI enrichers if enabled).
23. Match: compute match candidates and scores; create duplicate groups; store matching metadata and explainability.
24. Stewardship: stewards resolve duplicates and exceptions; overrides are tracked with before/after values and justification.
25. Consolidation: apply survivorship; generate master and golden records; maintain history tables and soft-deletes.
26. Post-validation: validate golden; quarantine invalid goldens; keep audit trail intact.
27. Publish: expose GD data via APIs and named queries; provide exports; maintain consistent versioned schemas.
28. Notify: emit events and notifications; route to webhook/email/brokers with retries and observability.
29. Monitor: job dashboard, SLA and queue monitoring, audit logs, and alerting.

5.3 Data Views and Physical Tables (core nuance)

Parity requirement: multiple lifecycle tables per entity are mandatory for correctness and auditability. GD should be the primary consumption view for certified data. History and soft-delete logs must be available for compliance.

View / table prefix	Meaning
SA	Source Authoring (manual / override authoring)
SD	Publisher Source Data (loaded)
AE/SE/GE	Error tables (authoring/source/golden)
MD/MH/MX	Master data + history + soft deletes

GD/GH/GX	Golden data + history + soft deletes
DU/UM	Duplicate management tables for fuzzy match and steward decisions
GA/GF/GP/SF	Override + flag tables tracking overridden attributes and original values

6. Feature Catalog (Semarchy-class parity)

6.1 Studio (Design-Time)

- Model Designer (entities, relationships, reference data, hierarchy definitions).
- Rule Builder (validations, enrichers, match rules, survivorship, overrides).
- Application Builder (business views, forms, navigation, dashboards).
- Workflow Builder (design, routing, SLAs, escalations).
- API Builder (named queries, endpoint packaging, access scopes).
- Environment Promotion (versioning, migrations, rollback).

6.2 Runtime Applications

- Data browsing/search (by entity, by relationship, by hierarchy).
- Data authoring (create/update, bulk edits, controlled fields).
- Duplicate Manager (queue, explainable match scores, merge/split).
- Exception Management (error queues, recycle, root cause tracking).
- My Tasks (workflow workbench, approvals, comments).
- Audit/History views (timeline, field-level provenance).

6.3 Integration and APIs

- REST CRUD + search + batch endpoints.
- SQL API for high-throughput loads and queries.
- Named Queries (custom read endpoints and notification payload builders).
- Exports/imports (CSV/Excel) with mapping and validation feedback.
- SDKs/clients (future).

6.4 Platform services

- Notification Service (webhook/email/Kafka/JMS targets).
- Plugin runtime (validators/enrichers) with sandboxing and credential management.
- Security layer (SSO, RBAC, record/attribute permissions, audit logs).
- Operations plane (monitoring, alerting, backup/restore, upgrades).

7. Requirements

7.1 Functional Requirements (detailed)

ID	Module	Requirement	Acceptance Criteria (DoD)	Priority
FR-001	Modeling	Create/edit entities, attributes, complex types, relationships; choose entity strategy.	Model saved, versioned; schema diff generated; validation errors shown.	P0
FR-002	Schema	Generate lifecycle tables and system columns per entity (SA/SD/MD/GD + history/errors/dup).	DDL generated; applied in DB; migrations repeatable and reversible.	P0
FR-003	Integration	Bulk load with Load ID init/submit/cancel; idempotent operations.	Loads can be retried safely; partial failures isolated; metrics available.	P0
FR-004	Quality	Validation engine with rule library; error tables and recycle workflow.	Errors visible; can be corrected and reprocessed; audit retained.	P0
FR-005	Enrichment	Enrichment pipeline (pre/post) + plugin hooks.	Built-in enrichers run; plugin enrichers can be registered and versioned.	P0

FR-006	Match/Merge	Fuzzy matching, scoring, grouping; steward review queue.	Match explanations; DU/UM tables; merge and split supported.	P1
FR-007	Consolidation	Survivorship policies, overrides with justification; golden generation.	Golden attributes show provenance; overrides tracked; history maintained.	P1
FR-008	Runtime UI	Business views, search, forms, parent-child navigation.	Configurable UI; permissions enforced; usable by non-technical users.	P0
FR-009	Duplicate Manager	Side-by-side compare; merge/split; policy-driven approvals.	Decisions audited; workflows supported; reprocessing works.	P1
FR-010	Named Queries	Define and expose parameterized named queries (custom endpoints).	Endpoints versioned; tested; access scopes; performant queries.	P1
FR-011	Workflow	Workflow designer + My Tasks UI + SLA timers.	Routing works; escalations; reminders; audit trail; APIs.	P1
FR-012	Notifications	Change-triggered notifications with retries and monitoring.	Webhook/email in v1.0; broker targets later; dead-letter handling.	P1

FR-013	Security	SSO (OIDC/SAML), RBAC, record/attribute permissions, audit logs.	Verified via tests; admin UI manages roles; logs immutable.	P0
FR-014	Ops	Job monitoring dashboards and alerts.	Loads, certifications, notifications, workflows visible with statuses.	P0

7.2 Non-Functional Requirements

Security: Encryption at rest and in transit; secret vault integration; secure plugin sandbox; audit logs immutable.

Performance: Scale to millions of records with incremental runs; batch parallelism; index guidance and auto-tuning hooks.

Reliability: Idempotent loads; replay-safe notifications; job resumption; clear states and retries.

Observability: Structured logs + metrics + traces; per-load/per-record lineage; dashboards and alerts.

Extensibility: Versioned plugins; stable APIs; schema evolution with backward compatibility guarantees.

Compliance: Audit, history, retention, masking, and export controls for regulated use cases.

8. API Requirements (REST + SQL + Named Queries)

8.1 REST API principles

- Versioned endpoints with backward compatibility rules (major version breaks only).
- Consistent pagination, sorting, filtering, and field selection.
- Strict auth scopes + rate limiting; tenant isolation.
- Bulk endpoints for upsert and batch delete with validation feedback.

8.2 Named Queries

Named queries are first-class artifacts: parameterized, versioned, access-controlled queries that produce stable response shapes. They are used both for custom read endpoints and as payload builders for notifications.

8.3 SQL API

SQL API supports high-throughput batch load patterns. It must provide Load ID lifecycle management, validation/execution status, and safe retry semantics.

9. Matching, Merge, and Survivorship

9.1 Matching types

- ID match (deterministic): match on strong identifiers (e.g., national ID, external key).
- Fuzzy match: configurable comparators (string similarity, phonetics, token sets), weighted scoring, thresholds.
- Grouping: create duplicate groups and candidate pairs; store match explanations for steward transparency.

9.2 Merge + survivorship

- Survivorship rules: source priority, recency, completeness, confidence score, and steward override.
- Override tracking: record which fields were overridden and keep prior values plus justification.
- History: maintain master and golden history for audit and rollback of decisions.

10. Security, Privacy, and Compliance

10.1 Authentication and identity

- OIDC for SSO (primary), SAML (enterprise), local users for break-glass.
- External directory sync optional (SCIM) for SaaS.

10.2 Authorization

- RBAC for roles; permissions on entity/actions.
- Record-level security (row filters by domain, organization, facility, etc.).
- Attribute-level controls (hide/mask/write-protect specific fields).

10.3 Audit and data governance

- Immutable audit log for API calls, UI actions, and background jobs.
- Field-level provenance (which source contributed to golden attribute).
- Retention policies and soft delete workflows.

11. Deployment and Operations

11.1 Supported deployment modes

- Self-hosted: single tenant; Postgres; VM or container.

- Kubernetes: HA services; horizontal scaling; external DB; object storage for artifacts.
- Cloud SaaS: multi-tenant; tenant isolation; managed upgrades.
- Roadmap: warehouse-native execution.

11.2 Observability

- Job console (loads, certification runs, match runs, workflows, notifications).
- Metrics: throughput, latency, error rates, duplicate rate, merge volume, task SLA compliance.
- Alerts: stuck jobs, high error rate, notification delivery failures, auth anomalies.

12. Success Metrics

Metric	How measured	Target (pilot)
Duplicate rate reduction	Before/after duplicates per 1,000 records	>= 60% reduction
Certification SLA	Time from ingest to GD publish	< 2 hours batch; < 15 min incremental
Steward productivity	Duplicates resolved per steward per day	2x baseline
Downstream sync reliability	Notification delivery success and retries	>= 99.5% success
Audit completeness	Coverage of audited actions	100% of write actions audited

13. Release Plan (high-level)

MVP (v0.9)

- Design-time model designer; Postgres schema generator; batch load; validation; basic enrichment; browse/search UI; audit/history; REST + SQL read patterns.

v1.0

- Fuzzy match/merge and duplicate manager; survivorship and overrides; named queries; webhook/email notifications; role-based permissions hardened.

v1.1

- Workflow builder + My Tasks; broker notifications (Kafka/JMS); plugin packaging/marketplace; AI enrichers and classification; improved admin tooling.

v1.2+

- Multi-tenant SaaS; warehouse-native engine; catalog/metadata harvesting; CDC change feed; advanced streaming.

14. Risks and Open Questions

Scope parity risk: Semarchy-class parity covers a wide surface area. Mitigation: phased delivery, feature flags, and strict MVP boundaries.

Matching accuracy: Need evaluation datasets and tuning tooling. Mitigation: match explainability + offline evaluation harness.

Schema evolution: Model changes can break downstream. Mitigation: versioned APIs + controlled migrations.

Plugin safety: Plugins can introduce security risks. Mitigation: sandbox + allowlists + signing + audit.

15. References (public Semarchy documentation used as parity anchors)

- Named Queries (custom REST endpoints; multi-view execution): https://docs.semarchy.com/sdp/reference/sdp/api/data-advanced/named-queries
- Data tables reference (SA/SD/MD/GD/DU/UM etc.): https://docs.semarchy.com/sdp/reference/dm/data-table-structures/data-tables
- Data table columns (built-in system columns): https://docs.semarchy.com/sdp/reference/dm/data-table-structures/data-table-columns
- Plugin enrichers list (AI + traditional enrichers): https://docs.semarchy.com/sdp/reference/dm/plugin-list
- Plugin validators list: https://docs.semarchy.com/sdp/reference/dm/validator-list
- Business views/hierarchies tutorial: https://www.semarchy.com/tutorials-content/business-views-hierarchies/
- Data notifications using named queries (support): https://support.semarchy.com/support/solutions/articles/43000727067-how-to-create-a-parent-based-data-notification-triggered-on-child-update
- Warehouse-native positioning (Snowflake): https://semarchy.com/blog/building-certified-trusted-data-natively-in-snowflake-with-semarchy-xdm/